

OVERFLOW RULE

// If any value or sum/product can exceed $\sim 2 \times 10^9$, use long long

FAST I/O

```
#define SpeedxKH ios_base::sync_with_stdio(false); cin.tie(NULL);
```

2D VECTOR TRAVERSE

```
for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) cout << g[i][j] << " ";
```

VECTOR – DECLARATION

```
vector<int> v; // empty vector
```

```
vector<int> v(n); // size n, garbage vector<int> v(n, 0); // size n, all 0
```

```
vector<vector<int>> g(n, vector<int>(m, 0)); // 2D vector (n x m)
```

VECTOR – INPUT / OUTPUT

```
for (int i=0;i<n;i++) cin>>v[i];
```

```
for (int x:v) cout<<x<<" ";
```

VECTOR – MODIFY

```
v.push_back(x); // add to end
```

```
v.pop_back(); // remove last
```

```
v.insert(v.begin()+pos, x); //insert at pos v.erase(v.begin()+pos); //erase at pos
```

```
v.clear(); // remove all
```

```
v.resize(newSize); // resize vector
```

```
v.size(); // element count
```

```
v.empty(); // true if size==0
```

```
v.front(); // first element
```

```
v.back(); // last element
```

VECTOR – MAX / MIN / SUM

```
*max_element(v.begin(),v.end()); //max value
```

```
*min_element(v.begin(),v.end()); //min value
```

```
max_element(v.begin(),v.end())-v.begin(); // index of max
```

```
min_element(v.begin(),v.end())-v.begin(); // index of min
```

```
accumulate(v.begin(), v.end(), 0LL); // sum, avoid overflow
```

VECTOR – SORT / REVERSE / SEARCH

```
sort(v.begin(), v.end()); // ascending
```

```
sort(v.begin(), v.end(), greater<int>()); // descending
```

```
reverse(v.begin(),v.end()); // reverse order
```

```
count(v.begin(),v.end(),x); // count of x
```

```
find(v.begin(), v.end(), x); // iterator (==v.end() if not found)
```

VECTOR – UNIQUE (remove duplicates, needs sorted)

```
sort(v.begin(),v.end()); v.erase(unique(v.begin(),v.end()),v.end());
```

SORT INDICES BY VALUE

```
vector<int> idx(n); iota(idx.begin(),idx.end(),0);
```

```
sort(idx.begin(),idx.end(),[&](int i,int j){return a[i]<a[j];});
```

EXTRA VECTOR ESSENTIALS

```
v1.insert(v1.end(),v2.begin(),v2.end()); // merge v2 into v1
```

```
swap(v1, v2); // O(1) swap
```

```
vector<int> v = {1, 2, 3}; // direct init
```

```
vector<int> v(v2.begin(), v2.end()); // copy from another vector
```

```
find(v.begin(), v.end(), x) != v.end(); // true if x exists
```

```
v.erase(remove(v.begin(), v.end(), x), v.end()); // remove all occurrences of x
```

```
fill(v.begin(), v.end(), x); // set all elements to x
```

```
#define ll long long #define ull unsigned long long #define ld long double
```

```
#define endl '\n'
```

```
const int MOD = 1e9 + 7;
```

```
const int INF = 1e9;
```

```
const ll LLINF = 1e18;
```

STRINGS – INPUT & BASICS

```
cin >> s; // single word  
s.length(); // or s.size()  
getline(cin, s); // full line w/ spaces  
s.empty(); // true if s==""
```

TRAVERSE

```
for (int i=0;i<s.size();i++) cout<<s[i]; for (char c : s) cout<<c;
```

CASE CONVERSION

```
s[i] = toupper(s[i]); s[i] = tolower(s[i]);  
transform(s.begin(), s.end(), s.begin(), ::toupper); // whole string upper  
transform(s.begin(), s.end(), s.begin(), ::tolower); // whole string lower
```

SUBSTRING & SEARCH

```
s.substr(pos, len); // from pos, len chars s.substr(pos); // from pos to end  
s.find("abc"); // index or string::npos  
if (s.find('c') != string::npos) // found check
```

COMPARE & CONCATENATE

```
s1 == s2; // equality s1 < s2; // lexicographic  
string r = s1 + s2; // concat s += "abc"; // append
```

INSERT / ERASE

```
s.insert(pos, "abc"); s.erase(pos, len); // erase len chars from  
pos
```

CHARACTER CHECKS

```
isalpha(c); isdigit(c); isupper(c); islower(c); isalnum(c); isspace(c);
```

CHAR / NUMBER CONVERSION

```
int d = s[i] - '0'; // char to digit char c = '0' + d; // digit to char  
int x = stoi(s); // string to int string s = to_string(x); // number to  
string  
long long x = stoll(s); // string to long long
```

FREQUENCY COUNT

```
int freq[26] = {0}; for (char c : s) freq[c-'a']++; // letter frequency
```

PALINDROME CHECK

```
string t = s; reverse(t.begin(), t.end()); bool isPalin = (s == t); // reverse  
method  
for (int i=0,j=s.size()-1; i<j; i++,j--) if (s[i]!=s[j]) { /*not palin*/ } // two-  
pointer method
```

ANAGRAM CHECK

```
string a=s1,b=s2; sort(a.begin(),a.end()); sort(b.begin(),b.end()); bool  
isAnagram=(a==b);
```

SPLIT BY DELIMITER

```
vector<string> tokens; stringstream ss(s); string tok;  
while (getline(ss, tok, ' ')) tokens.push_back(tok); // split on space
```

SORT CHARACTERS OF A STRING

```
sort(s.begin(), s.end());
```

BUILD REPEATED-CHAR STRING & TRIM SPACES

```
string s(n, 'a'); // string of n 'a' characters  
s.erase(remove(s.begin(), s.end(), ' '), s.end()); // remove all spaces
```

FREQUENCY (GENERAL / NUMERIC)

```
int freq[100005] = {0}; freq[x]++; // bounded values, fast array
map<int,int> freq; freq[x]++; // sparse/large values
unordered_map<int,int> freq; freq[x]++; // faster, unordered
int mx = 0, val = -1; for (auto &[k,v] : freq) if (v > mx) { mx = v; val = k; }
// most frequent
if (freq.count(x)) { /* x exists */ } // check existence in map
if (freq.count(target-x)) { /* pair found */ } // count pairs with sum = target
set<int> s(v.begin(), v.end()); s.size(); // count of distinct elements
// two-sum, return indices
map<int,int> seen; for(int i=0;i<n;i++){ if(seen.count(t-a[i])) {/*pair: seen[t-
a[i]],i*/} seen[a[i]]=i; }
```

SUBARRAY SUM = K (PREFIX SUM + FREQ MAP)

```
map<long long,int> cntPre; cntPre[0] = 1; long long pre = 0, ans = 0;
for (int x : a) { pre += x; ans += cntPre[pre - k]; cntPre[pre]++; } // count
subarrays summing to k
```

SORTING – CUSTOM COMPARATORS

```
sort(vp.begin(), vp.end(), [](pair<int,int> a, pair<int,int> b){ return a.second <
b.second; }); // sort pairs by 2nd
sort(words.begin(), words.end(), [](string a, string b){ return a.size() < b.size();
}); // sort by length
stable_sort(v.begin(), v.end()); // keeps equal order partial_sort(v.begin(),
v.begin()+k, v.end()); // sorts first k only
sort(v.rbegin(), v.rend()); // descending, alt to greater<int>()
sort(vp.begin(), vp.end(), [](pair<int,int> a, pair<int,int> b){ if(a.first!=b.first)
return a.first<b.first; return a.second>b.second; }); //tie-break: 1st asc, 2nd desc
```

BASIC MATH

```
abs(x); // absolute value
max({a,b,c}); min({a,b,c}); // multi-arg max/min
((a % m) + m) % m; // fix negative modulo
(a + b - 1) / b; // ceil division a/b
n*(n+1)/2; // sum 1..n n*(n+1)*(2*n+1)/6; // sum of squares 1..n
(x2-x1)*(x2-x1) + (y2-y1)*(y2-y1); // squared distance (avoid sqrt/float)
```

```
int mx=INT_MIN, mx2=INT_MIN; for(int x:a){ if(x>mx){mx2=mx;mx=x;}
else if(x>mx2 && x!=mx) mx2=x; } // 2nd largest, single pass
```

```
long long res=1; for(int i=0;i<b;i++) res*=a; // a^b, manual (avoid pow() float)
```

```
is_sorted(v.begin(), v.end()); // true if already sorted
```

ARITHMETIC PROGRESSION SUM

```
Sn = n/2 * (a + l);
Sn = n/2 * (2*a + (n-1)*d);
// Sn = sum of first n terms
// a = first term
// l = last term
// d = common difference
// n = number of terms
```

PAIR & TUPLE

```
pair<int,int> p = {1, 2}; p.first; p.second;           // 1, 2
pair<int,int> p2 = make_pair(1,2);                   // alternative syntax
auto [a, b] = p;                                     // structured binding
// Pairs compare lexicographically by default
pair<int,int> x = {1,5}, y = {1,3};
x > y;                                                // true (1==1, compares 5 vs 3)
```

```
vector<pair<int,int>> vp;
vp.push_back({5, 0});                                // {value, index}
sort(vp.begin(), vp.end());                           // by .first, ties by .second
sort(vp.rbegin(), vp.rend());                         // descending
swap(p, p2);                                          // swap two pairs
```

```
tuple<int,int,int> t = {1, 2, 3};
get<0>(t);                                            // access by position
tie(a, b, c) = t;                                    // unpack tuple
```

//Index of the maximum element

```
int maxIdx = max_element(a.begin(), a.end()) - a.begin();
```

MAP / UNORDERED MAP

map = sorted by key, $O(\log n)$. unordered_map = no order, $O(1)$ average (faster).

```
map<int,int> mp;
mp[key]++;                                           // insert if new or increment
mp[key] = 5;                                         // directly set value
mp.at(key);                                          // no auto insert (throws if missing)
```

```
mp.count(key);                                       // 1 if exists, 0 if not
mp.find(key) != mp.end();                           // alternative existence check
mp.erase(key);                                       // remove by key
mp.erase(mp.find(key));                             // remove by iterator
```

```
for (auto &[k, v] : mp) { }                        // iterate (sorted key order)
for (auto it = mp.rbegin(); it != mp.rend(); it++) { } // reverse iterate
```

```
mp.size(); mp.empty(); mp.clear();
mp.lower_bound(key);                                // first key >= key
mp.upper_bound(key);                                // first key > key
mp.begin()->first;                                   // smallest key
mp.begin()->second;                                  // value of smallest key
mp.rbegin()->first;                                  // largest key
mp.rbegin()->second;                                 // value of largest key
unordered_map<int,int> ump;                          // same syntax, unordered
map<string,int> wordCount;                             // string keys
map<pair<int,int>,int> grid;                           // 2D coordinate frequency
```

// Check if two arrays have the same frequency

```
map<int,int> cnt;
for (int x : a) cnt[x]++;
for (int x : b) cnt[x]--;
bool same = true;
for (auto &[k, v] : cnt)
    if (v != 0) same = false;
```

SET / UNORDERED SET

```
set = sorted,  $O(\log n)$ . unordered_set = no order,  $O(1)$  average (faster).
set<int> s;
s.insert(x); // adds x (ignored if already present)
auto res = s.insert(x); // res.second = true if x was newly inserted
s.count(x); // 1 if exists, 0 if not
s.erase(x); // remove by value
s.erase(s.find(x)); // remove by iterator
*s.begin(); // smallest element
*prev(s.end()); // largest element
s.lower_bound(x); // iterator to first element  $\geq x$ 
s.upper_bound(x); // iterator to first element  $> x$ 
s.size(); s.empty(); s.clear();
```

```
unordered_set<int> us; // same syntax, faster, unordered
// Check if array contains all numbers 1..n exactly once
set<int> s(a.begin(), a.end());
bool valid = (s.size() == n) &&
              (*s.begin() == 1) &&
              (*prev(s.end()) == n);
```

MULTISET

Like set, but allows duplicate values while staying sorted.

```
multiset<int> ms;
ms.insert(x); // allows duplicates
ms.count(x); // how many times x appears
ms.find(x) != ms.end(); // existence check
ms.erase(ms.find(x)); // erase ONE occurrence
ms.erase(x); // erases ALL occurrences of x
*ms.begin(); // smallest element
*prev(ms.end()); // largest element
ms.lower_bound(x); // first element  $\geq x$ 
ms.upper_bound(x); // first element  $> x$ 
ms.size(); ms.empty(); ms.clear();
// Maintain a running sorted multiset and always get current min/max
multiset<int> ms;
ms.insert(5); ms.insert(3); ms.insert(5);
int mn = *ms.begin(); // 3
int mx = *prev(ms.end()); // 5
ms.erase(ms.find(5)); // removes just ONE of the two 5's
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template<typename T>
using ordered_set =
tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
template<typename T>
using ordered_multiset =
tree<T, null_type, less_equal<T>, rb_tree_tag, tree_order_statistics_node_update>;
```

STACK (LIFO – Last In, First Out)

Use when: reversing order of arrival, matching brackets, undo operations, iterative DFS.

```
stack<int> st;
st.push(x);                // add to top
st.top();                  // peek top (doesn't remove)
st.pop();                  // remove top
st.empty(); st.size();

// Check if brackets are balanced
stack<char> st;
bool valid = true;
for (char c : s) {
    if (c == '(') st.push(c);
    else if (c == ')') {
        if (st.empty()) { valid = false; break; }
        st.pop();
    }
}
if (!st.empty()) valid = false; // unmatched opening brackets left
```

QUEUE (FIFO – First In, First Out)

Use when: processing in arrival order – BFS is the main use case.

```
queue<int> q;
q.push(x);                // add to back
q.front();                 // peek front
q.back();                  // peek back
q.pop();                   // remove front
q.empty(); q.size();
```

// Basic BFS traversal

```
queue<int> q;
vector<bool> visited(n, false);
q.push(start);
visited[start] = true;

while (!q.empty()) {
    int cur = q.front(); q.pop();
    for (int next : adj[cur]) {
        if (!visited[next]) {
            visited[next] = true;
            q.push(next);
        }
    }
}
```

DEQUE (double-ended queue – useful for sliding window)

```
deque<int> dq;
dq.push_back(x); dq.push_front(x);
dq.pop_back(); dq.pop_front();
dq.front(); dq.back();
dq.empty(); dq.size();
```

PAGE 7 – PREFIX SUM (1D)

Use when: repeated range-sum queries – $O(n)$ to build, $O(1)$ per query.

```
vector<long long> pre(n + 1, 0);
for (int i = 0; i < n; i++) pre[i + 1] = pre[i] + a[i];
```

```
// Sum of a[l..r] inclusive (0-indexed)
long long rangeSum = pre[r + 1] - pre[l];
```

```
// Demo – answer Q range-sum queries in  $O(1)$ 
vector<long long> pre(n + 1, 0);
for (int i = 0; i < n; i++) pre[i + 1] = pre[i] + a[i];
while (q--) {
    int l, r;
    cin >> l >> r;
    cout << pre[r + 1] - pre[l] << "\n";
}
```

```
// Prefix Frequency – count subarrays with sum = K
map<long long, int> freq;
freq[0] = 1;
long long sum = 0;
int cnt = 0;
for (int x : a) {
    sum += x;
    cnt += freq[sum - k];
    freq[sum]++;
}
```

```
// Prefix XOR
vector<int> preXor(n + 1, 0);
for (int i = 0; i < n; i++)
    preXor[i + 1] = preXor[i] ^ a[i];

// XOR of a[l..r]
int rangeXor = preXor[r + 1] ^ preXor[l];
```

PAGE 8 – TWO POINTER + SLIDING WINDOW

TWO POINTER

Use when: array is sorted (or has a monotonic property) – avoids $O(n^2)$ brute force.

```
// Two Sum (Opposite Direction)
int l = 0, r = n - 1;
while (l < r) {
    if (a[l] + a[r] == target) { /* found */ break; }
    else if (a[l] + a[r] < target) l++;
    else r--;
}
```

```
// Count pairs with sum < target
```

```

int l = 0, r = n - 1, count = 0;
while (l < r) {
    if (a[l] + a[r] < target) {
        count += (r - l);
        l++;
    }
    else r--;
}

```

// Remove duplicates (Same Direction)

```

int slow = 0;
for (int fast = 1; fast < n; fast++) {
    if (a[fast] != a[slow]) {
        slow++;
        a[slow] = a[fast];
    }
}
// a[0..slow] now contains unique elements

```

SLIDING WINDOW

// Fixed-size window

```

long long sum = 0;
for (int i = 0; i < k; i++) sum += a[i];
long long best = sum;
for (int i = k; i < n; i++) {
    sum += a[i] - a[i - k];
    best = max(best, sum);
}

```

// Smallest subarray with sum >= target

```

int l = 0;
long long sum = 0;
int best = INT_MAX;
for (int r = 0; r < n; r++) {
    sum += a[r];
    while (sum >= target) {
        best = min(best, r - l + 1);
        sum -= a[l++];
    }
}

```

// Longest substring without repeating characters

```

unordered_set<char> window;
int l = 0, best = 0;
for (int r = 0; r < s.size(); r++) {
    while (window.count(s[r])) {
        window.erase(s[l++]);
    }
    window.insert(s[r]);
    best = max(best, r - l + 1);
}

```


PREFIX SUM (2D)

```
vector<vector<long long>> pre(n+1, vector<long long>(m+1, 0));
for (int i=0;i<n;i++) for (int j=0;j<m;j++)
    pre[i+1][j+1] = grid[i][j] + pre[i][j+1] + pre[i+1][j] - pre[i][j];
long long rectSum = pre[r2+1][c2+1]-pre[r1][c2+1]-pre[r2+1][c1]+pre[r1][c1];
// (r1,c1)-(r2,c2)
```

Demo: answer Q rectangle-sum queries in O(1) each

```
while (q--) { cin>>r1>>c1>>r2>>c2;
    cout << pre[r2+1][c2+1]-pre[r1][c2+1]-pre[r2+1][c1]+pre[r1][c1] << "\n"; }
```

DIFFERENCE ARRAY

```
// size n+1 - the extra slot at r+1 cancels the update out
vector<long long> diff(n+1, 0);          diff[1]+=x; diff[r+1]-=x; // add x to [1,r]
long long cur=0; for(int i=0;i<n;i++){ cur+=diff[i]; a[i]=cur; } // reconstruct
```

Demo: apply Q range updates, then output final array

```
while (q--) {
    cin>>l>>r>>x; diff[l]+=x; diff[r+1]-=x;
}
long long cur=0; for(int i=0;i<n;i++){ cur+=diff[i]; cout<<cur<<" "; }
```

BINARY SEARCH (basic)

```
int l=0, r=n-1, ans=-1;
while (l <= r) {
    int mid = l + (r-l)/2;
    if (a[mid]==target) { ans=mid; break; }
    else if (a[mid]<target) l=mid+1;
    else r=mid-1;
}
binary_search(a.begin(), a.end(), target); // built-in, returns true/false only
```

LOWER BOUND / UPPER BOUND

```
lower_bound(a.begin(),a.end(),x); // first elem >= x
upper_bound(a.begin(),a.end(),x); // first elem > x
auto it = lower_bound(a.begin(), a.end(), x);
if (it != a.end()) cout << *it; // direct iterator use, dereference for value
int idx = lower_bound(a.begin(), a.end(), x) - a.begin(); // convert to index
bool exists = (idx < n && a[idx] == x); // check existence
int cnt = upper_bound(a.begin(),a.end(),x) - lower_bound(a.begin(),a.end(),x);
// count of x
```

Demo: find the smallest element >= x in a sorted array

```
auto it = lower_bound(a.begin(), a.end(), x);
if (it != a.end()) cout << *it; else cout << "no such element";
```

===== MULTIDIMENSIONAL LOWER BOUND (on vector of pairs, sorted) =====

```
lower_bound(vp.begin(), vp.end(), make_pair(x, INT_MIN));
// first pair with .first >= x
```

// Demo: count how many elements in sorted array are <= x

```
int cntLE = upper_bound(a.begin(), a.end(), x) - a.begin();
```

// Demo: count how many elements in sorted array are in range [lo, hi]

```
int cntRange = upper_bound(a.begin(),a.end(),hi) - lower_bound(a.begin(),a.end(),lo);
```

PRIORITY QUEUE (heap)

```
priority_queue<int> pq; // max-heap default
pq.push(x); pq.top(); pq.pop(); // top()=largest
priority_queue<int, vector<int>, greater<int>> minpq; // min-heap version
priority_queue<pair<int,int>> pq2; // max-heap of pairs, compares .first then .second
pq2.push({5,1}); pq2.push({3,2}); pq2.push({5,0}); pq2.top(); //{5,1} – highest first
// min-heap of pairs
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> minpq2;
```

Demo: find the k largest elements in an array

```
priority_queue<int, vector<int>, greater<int>> minHeap;
for (int x : a) { minHeap.push(x); if (minHeap.size() > k) minHeap.pop(); }
// minHeap now holds the k largest elements of a[]
```

Demo: merge top elements from multiple sorted streams

```
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
// {value, streamIndex}
for (int i=0;i<k;i++) pq.push({streams[i][0], i});
while (!pq.empty()) { auto [val,idx]=pq.top(); pq.pop();
/* push next from stream idx */ }
```

RECURSION

```
// every recursive function needs a base case – without one it recurses
// forever and crashes (stack overflow)
int solve(int n) {
    if (n == 0) return baseCaseValue; // base case – always needed
    return combine(n, solve(n - 1)); // recursive call on smaller input
}
```

Demo: factorial

```
long long fact(int n) { if (n<=1) return 1; return n*fact(n-1); }
```

Demo: Fibonacci with memoization (avoids exponential blowup)

```
map<int,long long> memo;
long long fib(int n) {
    if (n<=1) return n;
    if (memo.count(n)) return memo[n];
    return memo[n] = fib(n-1) + fib(n-2);
}
```

Demo: recursive sum of digits

```
int digitSum(int n) { if (n==0) return 0; return n%10 + digitSum(n/10); }
```

Demo: recursive binary search (recursion applied to searching)

```
int bsearch(vector<int>& a, int l, int r, int target) {
    if (l > r) return -1;
    int mid = l + (r-l)/2;
    if (a[mid] == target) return mid;
    else if (a[mid] < target) return bsearch(a, mid+1, r, target);
    else return bsearch(a, l, mid-1, target);
}
```

BASIC BACKTRACKING

// Try a choice, recurse, undo the choice (backtrack), try the next one.
// Used for generating subsets, permutations, or valid arrangements.

General template

```
void backtrack(/* state */) {  
    if (/* base case: solution complete */) {  
        // record/process the solution  
        return;  
    }  
    for (/* each possible choice */) {  
        // make choice  
        backtrack(/* updated state */);  
        // undo choice (backtrack)  
    }  
}
```

Demo 1: Generate all subsets of an array

```
vector<int> cur; vector<vector<int>> allSubsets;  
void gen(int idx, vector<int>& a) {  
    if (idx == a.size()) { allSubsets.push_back(cur); return; }  
    gen(idx + 1, a); // choice 1: don't include a[idx]  
    cur.push_back(a[idx]);  
    gen(idx + 1, a); // choice 2: include a[idx]  
    cur.pop_back(); // undo (backtrack)  
}
```

Demo 2: Generate all permutations of an array

```
void permute(vector<int>& a, int idx) {  
    if (idx == a.size()) { /* process a[] as one full permutation */ return; }  
    for (int i = idx; i < a.size(); i++) {  
        swap(a[idx], a[i]);  
        permute(a, idx + 1);  
        swap(a[idx], a[i]); // undo (backtrack)  
    }  
}  
// Alternative: use next_permutation in a loop instead of writing this by hand  
sort(a.begin(), a.end());  
do { /* use a */ } while (next_permutation(a.begin(), a.end()));
```

Demo 3: Combination Sum (pick numbers that sum to target, reuse allowed)

```
vector<int> cur; vector<vector<int>> results;  
void combSum(vector<int>& a, int idx, int remain) {  
    if (remain == 0) { results.push_back(cur); return; }  
    if (remain < 0 || idx == a.size()) return;  
    cur.push_back(a[idx]);  
    combSum(a, idx, remain - a[idx]); // reuse same element  
    cur.pop_back(); // undo (backtrack)  
    combSum(a, idx + 1, remain); // move to next element  
}
```

BINARY SEARCH ON ANSWER

```
int lo=minPossible, hi=maxPossible, ans=-1;
while (lo <= hi) {
    int mid = lo + (hi-lo)/2;
    if (isValid(mid)) { ans=mid; lo=mid+1; } // mid works, try bigger
    else hi = mid - 1; // mid fails, try smaller
}
```

Demo: minimum max value when splitting array into k parts

```
bool canSplit(vector<int>& a, int maxSum, int k) {
    int parts=1; long long cur=0;
    for (int x : a) { if (cur+x > maxSum) { parts++; cur=x; } else cur+=x; }
    return parts <= k;
}

int lo=*max_element(a.begin(),a.end()), hi=accumulate(a.begin(),a.end(),0), ans=hi;
while (lo <= hi) {
    int mid = lo + (hi-lo)/2;
    if (canSplit(a, mid, k)) { ans=mid; hi=mid-1; } else lo=mid+1;
}
```

Demo: smallest x such that $x*x \geq n$

```
long long lo=0, hi=n, ans=n;
while (lo <= hi) {
    long long mid = lo + (hi-lo)/2;
    if (mid*mid >= n) { ans=mid; hi=mid-1; } else lo=mid+1;
}
```

GCD / LCM

```
__gcd(a, b); // built-in GCD (also gcd(a,b) in C++17+)
long long lcm(long long a, long long b) { return a / __gcd(a,b) * b; } // divide
first, avoid overflow
```

How GCD actually works – iterative Euclidean Algorithm

```
long long gcdManual(long long a, long long b) {
    while (b != 0) { long long temp=b; b=a%b; a=temp; }
    return a;
}

long long g=a[0]; for(int i=1;i<n;i++) g=__gcd(g,a[i]); // GCD of array
long long l=a[0]; for(int i=1;i<n;i++) l=l/__gcd(l,a[i])*a[i]; // LCM of array
```

Demo: simplify fraction a/b

```
int g = __gcd(a, b); a /= g; b /= g;
```

===== GCD/LCM USEFUL FACTS =====

```
gcd(a, b) * lcm(a, b) == a * b; // always true – useful identity
gcd(a, 0) == a; // base case fact
gcd(a, b) == gcd(b, a % b); // Euclidean recurrence itself
// a and b are coprime (share no common factor) if gcd(a,b) == 1
bool coprime = (__gcd(a, b) == 1);
// n numbers are pairwise "reducible" to gcd=1 doesn't mean pairwise coprime
// (e.g. gcd(6,10,15)=1 but no two of them are individually coprime)
// divisibility check using gcd
bool aDividesB = (b % a == 0); // simple check, no gcd needed
// sum of two fractions a1/b1 + a2/b2, simplified
// numerator = a1*b2 + a2*b1, denominator = b1*b2, then divide both by gcd
long long num = a1*b2 + a2*b1, den = b1*b2;
long long g = __gcd(num, den); num/=g; den/=g;
```

PRIME CHECKING (trial division)

```
bool isPrime(long long n) {
    if (n < 2) return false;
    for (long long i=2; i*i<=n; i++) if (n%i==0) return false;
    return true;
}
```

Demo: count primes up to n (small n, no sieve)

```
int count=0; for(int i=2;i<=n;i++) if (isPrime(i)) count++;
// O(sqrt n) per check – fine for single queries. For n>=1e6+ or many
// queries, use Sieve of Eratosthenes (Rating 1200) instead.
```

MODULAR ARITHMETIC

// Always use long long before taking % to avoid overflow.

The three most common operations

```
(a + b) % MOD;
(a - b + MOD) % MOD; // add MOD to keep result non-negative
(a * b) % MOD;
const long long MOD = 1e9 + 7;
long long addMod(long long a, long long b) { return ((a%MOD)+(b%MOD))%MOD; }
long long subMod(long long a, long long b) { return ((a%MOD)-(b%MOD)+MOD)%MOD; }
long long mulMod(long long a, long long b) { return ((a%MOD)*(b%MOD))%MOD; }
```

Modular exponentiation (fast power) – $a^b \bmod MOD$

```
long long powMod(long long a, long long b, long long mod) {
    long long res=1; a%=mod;
    while (b > 0) {
        if (b & 1) res = res*a%mod;
        a = a*a%mod; b >>= 1;
    }
    return res;
}
```

Demo: modular inverse via Fermat's little theorem (mod must be prime)

```
long long modInverse(long long a, long long mod) { return powMod(a, mod-2, mod); }
// (a / b) % mod == (a * modInverse(b, mod)) % mod
```

Demo: compute $n! \bmod p$ for large n

```
long long fact=1; for(int i=1;i<=n;i++) fact=fact*i%MOD;
```

===== WHY DIVISION IS DIFFERENT UNDER MOD =====

```
(a / b) % MOD; // WRONG – division doesn't distribute over mod
(a * modInverse(b, MOD)) % MOD; //CORRECT – must multiply by modular inverse instead
```

===== MORE USEFUL MOD/PRIME NOTES =====

// modular exponentiation with negative exponent invalid, must use modInverse instead
// powMod(a, -1, mod) is WRONG – use modInverse(a, mod) for a^{-1}

// checking gcd(a, mod) == 1 before using modInverse (inverse only exists if coprime)

```
bool hasInverse = (__gcd(a, MOD) == 1);
```

// binomial coefficient $C(n, r) \bmod p$ – needs factorials + modInverse

```
long long nCr(int n, int r, vector<long long>& fact, long long mod) {
```

```
    if (r < 0 || r > n) return 0;
```

```
    return fact[n] * modInverse(fact[r], mod) % mod * modInverse(fact[n-r], mod) %
```

```
mod;
```

```
}
```

// precompute factorials up to N once, reuse for many nCr queries

```
vector<long long> fact(N+1); fact[0]=1;
```

```
for (int i=1;i<=N;i++) fact[i] = fact[i-1]*i%MOD;
```

XOR PROPERTIES

```
// a ^ a = 0          (anything XORed with itself is 0)
// a ^ 0 = a          (XOR with 0 changes nothing)
// XOR is commutative and associative – order doesn't matter
x ^= y; // toggles bits of x wherever y has a 1
Demo: find the single number that appears once (others appear twice)
int result=0; for(int x:a) result^=x; // pairs cancel via a^a=0
Demo: find the missing number in array of 0..n (one missing)
int missing=0;
for (int i=0;i<=n;i++) missing^=i; // XOR all numbers 0..n
for (int x:a) missing^=x; // XOR all array elements
// remaining value is the missing number
```

BIT MANIPULATION (basic)

```
(n >> i) & 1; // check if bit i is set
n | (1 << i); // set bit i to 1
n & ~(1 << i); // clear bit i (set to 0)
n ^ (1 << i); // toggle bit i
n & (n - 1); // removes the lowest set bit
n & (-n); // isolates the lowest set bit (lowbit)
n << 1; n >> 1; // multiply/divide by 2 (fast)
Demo: check if a number is a power of 2
bool isPowerOf2 = (n > 0) && ((n & (n-1)) == 0);
Demo: generate all subsets of a set using bitmasks (n up to ~20)
for (int mask=0; mask<(1<<n); mask++) {
    for (int i=0;i<n;i++) {
        if (mask & (1<<i)) { /* element i is in this subset */ }
    }
}
```

builtin popcount

```
__builtin_popcount(x); // number of set bits (int)
__builtin_popcountll(x); // same, for long long
__builtin_clz(x); // count leading zeros (int)
__builtin_ctz(x); // count trailing zeros – position of lowest set bit
Demo: count total set bits across an array
long long totalBits=0; for(int x:a) totalBits += __builtin_popcount(x);
```

===== BONUS: TWO numbers that appear once (all others appear twice) =====

```
// Demo: find both unique numbers using XOR + lowbit isolation
int xorAll = 0;
for (int x : a) xorAll ^= x; // xorAll = num1 ^ num2

int diffBit = xorAll & (-xorAll); // isolate one bit where num1 and num2 differ

int num1 = 0, num2 = 0;
for (int x : a) {
    if (x & diffBit) num1 ^= x; // group 1: bit is set
    else num2 ^= x; // group 2: bit is not set
}
// num1 and num2 are the two unique numbers
```

INTERMEDIATE CONSTRUCTIVE (math-based)

// Build a valid answer directly using a pattern – parity, modular
// structure, or a clever formula – instead of searching. The skill
// is spotting the right observation, not memorizing code.

Common Constructive Observations

Odd / Even (Parity) – does the answer depend on n being odd or even?

Prefix / Suffix – can you build left-to-right or right-to-left?

Reverse – does reversing the array/string reveal a pattern?

Rotation – does shifting elements cyclically help?

Alternating Pattern – $+1/-1$, high/low, or $0/1$ repeating structure

Cyclic Assignment – assign values in a repeating $1..k$ cycle

Greedy Construction – always take the locally valid choice

Smallest/Largest Valid Answer – construct the extreme case directly

Work Backwards – start from the target/end state, build toward input

Demo 1: Alternating pattern construction

```
vector<int> a(n); a[0] = 0;
for (int i=1;i<n;i++) a[i] = a[i-1] + (i%2==0 ? 1 : -1);
```

Demo 2: Cyclic assignment (repeating pattern of k values)

```
vector<int> a(n);
for (int i=0;i<n;i++) a[i] = (i % k) + 1;
```

Demo 3: Greedy digit-by-digit – smallest n -digit number summing to s

```
string result = ""; int remaining = s;
for (int i=0;i<n;i++) {
    int digitsLeft = n - i - 1;
    int d = max(0, remaining - digitsLeft*9); // smallest digit still completable
    d = max(i==0 ? 1 : 0, d); // no leading zero
    result += ('0' + d);
    remaining -= d;
}
```

// Before coding: try small cases ($n=1,2,3$) by hand and look for the
// pattern – most constructive problems fit one of the categories above.

// Demo 4: Work backwards – reconstruct array from given prefix XOR/sum
// (common pattern: you're given the "result" array, must recover original)

```
vector<int> pre = {0, 3, 1, 6}; // given prefix XOR values, pre[0]=0
vector<int> a(pre.size() - 1);
for (int i = 0; i < a.size(); i++)
    a[i] = pre[i] ^ pre[i+1]; // recover original element from consecutive
prefix values
```

// Demo 5: Parity-based pairing – pair smallest with largest (common constructive
trick)

```
sort(a.begin(), a.end());
vector<int> result;
int l = 0, r = n - 1;
while (l <= r) {
    result.push_back(a[l++]);
    if (l <= r) result.push_back(a[r--]);
}
```

SIEVE OF ERATOSTHENES

Use when: many primality checks/prime lists needed up to N.

```
const int MAXN = 1000005;
vector<bool> isComposite(MAXN, false); vector<int> primes;
void sieve() { // O(N log log N)
    for (int i=2; i<MAXN; i++) {
        if (!isComposite[i]) {
            primes.push_back(i);
            for (long long j=(long long)i*i; j<MAXN; j+=i) isComposite[j]=true;
        }
    }
}
bool isPrimeFast(int x) { return x >= 2 && !isComposite[x]; } // O(1) after sieve
```

PRIME FACTORIZATION

Use when: factorizing many numbers – precompute spf once.

```
vector<int> spf(MAXN);
void sieveSPF() { // smallest prime factor sieve
    for (int i=2; i<MAXN; i++) if (spf[i]==0)
        for (int j=i; j<MAXN; j+=i) if (spf[j]==0) spf[j]=i;
}
vector<int> factorize(int x) { // O(log n) using SPF
    vector<int> f; while (x > 1) { f.push_back(spf[x]); x /= spf[x]; }
    return f;
}
vector<pair<int,int>> factorizeBasic(int n) { // O(sqrt n), {prime, exponent}
    vector<pair<int,int>> res;
    for (int p=2; (long long)p*p<=n; p++) {
        if (n % p != 0) continue;
        int cnt=0; while (n % p == 0) { n /= p; cnt++; }
        res.push_back({p, cnt});
    }
    if (n > 1) res.push_back({n, 1});
    return res;
}
```

DIVISORS & MULTIPLES COUNTING

```
int countDivisors(int n) { // O(sqrt n)
    int cnt=0;
    for (int i=1; (long long)i*i<=n; i++) if (n%i==0) { cnt++; if (i!=n/i) cnt++; }
    return cnt;
}
vector<int> getDivisors(int n) { // O(sqrt n)
    vector<int> d;
    for (int i=1; (long long)i*i<=n; i++) if (n%i==0) { d.push_back(i); if (i!=n/i)
d.push_back(n/i); }
    return d;
}
```

Use when: divisor counts needed for every number up to N at once – $\sim O(N \log N)$.

```
vector<int> divCount(MAXN, 0); // O(N log N) precompute
for (int i=1; i<MAXN; i++) for (int j=i; j<MAXN; j+=i) divCount[j]++;

// #divisors =  $\prod(\text{exponent} + 1)$ 
```


BINARY EXPONENTIATION

Use when: computing a^b fast – $O(\log b)$ instead of $O(b)$. One of the most reused templates in CP.

// Core idea: $a^b = (a^2)^{(b/2)}$ if b even, $a * a^{(b-1)}$ if b odd
// halves the exponent each step instead of multiplying one at a time

BASIC (no mod)

```
long long power(long long a, long long b) {
    long long res = 1;
    while (b > 0) {
        if (b & 1) res *= a; // exponent odd: take current a
        a *= a; // square the base
        b >>= 1; // halve the exponent
    }
    return res;
}
```

MODULAR VERSION (links to powMod on Page 13)

```
long long powMod(long long a, long long b, long long mod) {
    long long res = 1; a %= mod;
    while (b > 0) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
```

RECURSIVE VERSION

```
long long powRec(long long a, long long b, long long mod) {
    if (b == 0) return 1;
    long long half = powRec(a, b/2, mod);
    long long res = half * half % mod;
    if (b & 1) res = res * a % mod;
    return res;
}
```

COMMON APPLICATIONS

`powMod(a, mod-2, mod);` // modular inverse (mod prime) – see Page 13
`powMod(2, n, mod);` // count of subsets, reachable states, etc.
`power(base, n);` // matrix exponentiation uses this same pattern on matrices
Key note: always use long long – intermediate squarings overflow int fast.

===== USEFUL FACTS =====

// $a^0 = 1$ (base case, always)
// $a^b * a^c = a^{(b+c)}$ (exponent addition rule)
// $(a^b)^c = a^{(b*c)}$ (exponent multiplication rule)
// $a^b \bmod p$, if $\gcd(a,p)=1$ and p prime: $a^{(p-1)} \equiv 1 \pmod p$ – Fermat's Little Theorem
// number of set bits in b = number of multiplications needed (rest are squarings)
// time complexity: $O(\log b)$ multiplications total
// $a^{(-1)} \bmod p == a^{(p-2)} \bmod p$ (modular inverse, p prime – see Page 13)
// b in binary determines which squarings get multiplied into result
// example: $b=13$ (1101_2) $\rightarrow a^{13} = a^8 * a^4 * a^1$
// overflow risk: $(a*a)$ can exceed long long if a is near $1e9+$ – reduce mod early
// `power(a, b)` with b negative is undefined here – handle sign separately if needed

WHAT DP IS

// Break a problem into overlapping subproblems, solve each ONCE, store the
// result (memoize), reuse it instead of recomputing. Two forms: top-down
// (recursion + memo) and bottom-up (iterative table fill).

1D DP – TOP-DOWN (memoization)

Use when: recursive structure is natural but overlapping calls repeat work.

```
vector<long long> memo(n+1, -1);  
long long solve(int i) {  
    if (i <= 0) return baseCase;  
    if (memo[i] != -1) return memo[i]; // already computed  
    return memo[i] = combine(solve(i-1), solve(i-2) /* etc */);  
}
```

1D DP – BOTTOM-UP (tabulation)

Use when: order of subproblems is clear – usually faster, avoids recursion overhead.

```
vector<long long> dp(n+1);  
dp[0] = base0; dp[1] = base1;  
for (int i = 2; i <= n; i++) dp[i] = combine(dp[i-1], dp[i-2]);
```

DEMO 1: FIBONACCI

```
vector<long long> dp(n+1); dp[0]=0; dp[1]=1;  
for (int i=2;i<=n;i++) dp[i] = dp[i-1] + dp[i-2];
```

DEMO 2: CLIMBING STAIRS (1 or 2 steps at a time)

```
vector<long long> dp(n+1); dp[0]=1; dp[1]=1; // dp[i] = ways to reach step i  
for (int i=2;i<=n;i++) dp[i] = dp[i-1] + dp[i-2];
```

DP ON ARRAYS

Use when: dp[i] depends on a[i] plus one or more previous dp states – 'take or skip' shape.

```
vector<long long> dp(n);  
dp[0] = a[0];  
for (int i=1;i<n;i++) dp[i] = max(dp[i-1], dp[i-1] + a[i]); // generic take/extend  
pattern
```

DEMO 3: MAXIMUM SUM / HOUSE ROBBER (no two adjacent chosen)

```
vector<long long> dp(n);  
dp[0] = a[0]; dp[1] = max(a[0], a[1]);  
for (int i=2;i<n;i++) dp[i] = max(dp[i-1], dp[i-2] + a[i]);  
// answer = dp[n-1] // dp[i] = best using first i+1 houses
```

SPACE OPTIMIZATION NOTE

```
// if dp[i] only needs dp[i-1] and dp[i-2], drop the array – keep 2 variables  
long long prev2=base0, prev1=base1, cur;  
for (int i=2;i<=n;i++) { cur = prev1 + prev2; prev2 = prev1; prev1 = cur; }
```

```
// ===== HOW TO RECOGNIZE A DP PROBLEM =====
```

```
// - "count the number of ways to..." → likely DP
```

```
// - "find min/max cost/sum/length to..." → likely DP
```

```
// - Brute force is exponential AND subproblems repeat → DP candidate
```

```
// - Can you define dp[i] = "best answer using first i elements"? → good sign
```

```
// ===== DP CHECKLIST =====
```

```
// 1. Define the state: what does dp[i] represent?
```

```
// 2. Find the transition: how does dp[i] relate to smaller states?
```

```
// 3. Base case: what is dp[0] (or dp[1])?
```

```
// 4. Order: fill dp left-to-right, or need recursion + memo?
```

```
// 5. Answer: is it dp[n], max(dp), or something derived from the array?
```

THE KNAPSACK FAMILY – SHARED SHAPE

```
// All three: for each item, either SKIP it or TAKE it, track a capacity/target.  
// dp[i][cap] = best answer using first i items with capacity/target cap  
// dp[i][cap] = combine(dp[i-1][cap] /*skip*/, dp[i-1][cap-w[i]] /*take*/)
```

0-1 KNAPSACK (each item used at most once)

Use when: maximize value under a weight limit, each item picked 0 or 1 times.

```
vector<vector<long long>> dp(n+1, vector<long long>(W+1, 0));  
for (int i=1;i<=n;i++) {  
    for (int cap=0;cap<=W;cap++) {  
        dp[i][cap] = dp[i-1][cap]; // skip item i  
        if (w[i-1] <= cap)  
            dp[i][cap] = max(dp[i][cap], dp[i-1][cap-w[i-1]] + val[i-1]); // take  
    }  
}  
// answer = dp[n][W]  
// 1D space-optimized: loop cap DESCENDING (prevents reusing item same pass)  
vector<long long> dp(W+1, 0);  
for (int i=0;i<n;i++) for (int cap=W;cap>=w[i];cap--) dp[cap]=max(dp[cap],dp[cap-  
w[i]]+val[i]);
```

COIN CHANGE (unlimited use of each coin)

Use when: fewest coins (or count ways) to make a target – coins reusable.

```
vector<int> dp(target+1, INT_MAX); // min coins to make each amount  
dp[0] = 0;  
for (int cap=1;cap<=target;cap++)  
    for (int c : coins) if (c<=cap && dp[cap-c]!=INT_MAX) dp[cap]=min(dp[cap],dp[cap-  
c]+1);  
// answer = dp[target] (or -1 if still INT_MAX)  
// NOTE: loop cap ASCENDING here – unlike knapsack, reuse is intended
```

SUBSET SUM (can a subset sum to exactly target?)

Use when: checking if some subset of elements sums to a target – true/false, each used once.

```
vector<bool> dp(target+1, false); dp[0] = true;  
for (int x : a)  
    for (int cap=target;cap>=x;cap--) dp[cap] = dp[cap] || dp[cap-x]; // descending:  
0-1 style  
// answer = dp[target]
```

QUICK COMPARISON

```
// 0-1 Knapsack : each item once      → loop capacity DESCENDING  
// Coin Change  : items reusable      → loop capacity ASCENDING  
// Subset Sum   : each item once, bool → loop capacity DESCENDING
```

===== HOW TO PICK THE RIGHT PATTERN =====

```
// "each item used at most once" + maximize value      → 0-1 Knapsack  
// "unlimited coins/items" + minimize count/ways        → Coin Change  
// "does a subset exist that sums to X" (true/false)   → Subset Sum  
// keyword "at most once" or "can't reuse"             → loop DESCENDING  
// keyword "unlimited" or "as many times as you want"   → loop ASCENDING
```

===== COMMON MISTAKE =====

```
// Using ascending loop for 0-1 Knapsack lets an item get counted twice  
// – always double check the loop direction matches the problem's reuse rule
```

GRAPH REPRESENTATION (Adjacency List)

```
vector<vector<int>>> adj(n); // unweighted
adj[u].push_back(v); adj[v].push_back(u); // undirected edge u-v
adj[u].push_back(v); // directed edge u->v only
vector<vector<pair<int,int>>>> wadj(n); // weighted: {neighbor, weight}
wadj[u].push_back({v, w});
```

DFS (Depth First Search)

Use: exploring as deep as possible before backtracking connectivity, cycles, paths.

```
vector<bool> visited(n, false);
void dfs(int u) {
    visited[u] = true;
    // process u here
    for (int v : adj[u]) if (!visited[v]) dfs(v);
}
// iterative version (avoids stack overflow on deep graphs)
void dfsIter(int start) {
    stack<int> st; st.push(start);
    while (!st.empty()) {
        int u = st.top(); st.pop();
        if (visited[u]) continue;
        visited[u] = true; // process u here
        for (int v : adj[u]) if (!visited[v]) st.push(v);
    }
}
```

BFS (Breadth First Search)

Use when: exploring level-by-level – shortest path in unweighted graphs.

```
vector<bool> visited(n, false); vector<int> dist(n, -1);
void bfs(int start) {
    queue<int> q; q.push(start);
    visited[start] = true; dist[start] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) if (!visited[v]) {
            visited[v] = true; dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
} // dist[v] after bfs = shortest #edges from start to v (unweighted only)
===== WEIGHTED EDGE LIST INPUT (common contest input format) =====
vector<vector<pair<int,int>>>> wadj(n);
for (int i=0;i<m;i++) {
    int u,v,w; cin>>u>>v>>w; u--; v--; // convert 1-indexed to 0-indexed
    wadj[u].push_back({v,w}); wadj[v].push_back({u,w});
}
===== BFS SHORTEST PATH RECONSTRUCTION =====
vector<int> parent(n, -1);
// inside bfs, when visiting v from u: parent[v] = u;
vector<int> path;
for (int v=target; v!=-1; v=parent[v]) path.push_back(v);
reverse(path.begin(), path.end());
// U R D L
int dx[] = {-1,0,1,0};
int dy[] = {0,1,0,-1};
```

CONNECTED COMPONENTS

Use when: counting/grouping separate 'islands' in a graph.

```
vector<bool> visited(n, false); int numComponents = 0;
for (int i=0;i<n;i++) {
    if (!visited[i]) { numComponents++; dfs(i); } // reuse dfs from Page 20
}
```

CYCLE DETECTION

Undirected graph: a visited neighbor that isn't the parent means a cycle.

```
bool hasCycleUndirected(int u, int parent, vector<bool>& vis) {
    vis[u] = true;
    for (int v : adj[u]) {
        if (!vis[v]) { if (hasCycleUndirected(v, u, vis)) return true; }
        else if (v != parent) return true;
    }
    return false;
}
```

Directed graph: track the current recursion path – a back-edge into it means a cycle.

```
vector<int> state; // 0=unvisited, 1=in progress, 2=done
bool hasCycleDirected(int u) {
    state[u] = 1;
    for (int v : adj[u]) {
        if (state[v] == 1) return true; // back-edge to in-progress node
        if (state[v] == 0 && hasCycleDirected(v)) return true;
    }
    state[u] = 2; return false;
}
```

BIPARTITE CHECK

Use when: checking if a graph's nodes can be 2-colored with no adjacent same color.

```
vector<int> color(n, -1);
bool isBipartite(int start) {
    queue<int> q; q.push(start); color[start] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (color[v] == -1) { color[v] = 1 - color[u]; q.push(v); }
            else if (color[v] == color[u]) return false;
        }
    }
    return true;
}
```

FLOOD FILL

Use when: filling/marking a connected region – grid version of DFS/BFS.

```
void floodFill(int r, int c, vector<vector<int>>& grid, int target, int newVal) {
    if (r<0||r>=grid.size()||c<0||c>=grid[0].size()||grid[r][c]!=target) return;
    grid[r][c] = newVal;
    floodFill(r+1,c,grid,target,newVal); floodFill(r-1,c,grid,target,newVal);
    floodFill(r,c+1,grid,target,newVal); floodFill(r,c-1,grid,target,newVal);
}
```

===== DIRECTED GRAPH INDEGREE =====

```
vector<int> indegree(n, 0);
for (int u=0; u<n; u++)
    for (int v : adj[u]) indegree[v]++;
```

TREE BASICS

A tree = connected graph with n nodes, n-1 edges, no cycles. Use adjacency list.

```
vector<vector<int>> adj(n); // same structure as a graph
vector<int> depth(n, 0), subtreeSize(n, 1);
void dfsTree(int u, int parent) {
    for (int v : adj[u]) {
        if (v == parent) continue; // skip edge back to parent
        depth[v] = depth[u] + 1;
        dfsTree(v, u);
        subtreeSize[u] += subtreeSize[v];
    }
}
// call dfsTree(root, -1) – root has no real parent
// leaf node: adj[u].size()==1 && u != root // diameter: 2x BFS/DFS from any node
```

GRID-BASED BFS / DFS

Use when: pathfinding or region exploration on a 2D grid – treat cells as graph nodes.

```
int dr[] = {-1,1,0,0}, dc[] = {0,0,-1,1}; // 4-directional moves
bool inBounds(int r,int c,int R,int C) { return r>=0&&r<R&&c>=0&&c<C; }
```

GRID BFS – shortest path in unweighted grid

```
vector<vector<int>> dist(R, vector<int>(C, -1));
queue<pair<int,int>> q;
dist[sr][sc] = 0; q.push({sr, sc});
while (!q.empty()) {
    auto [r, c] = q.front(); q.pop();
    for (int d=0;d<4;d++) {
        int nr=r+dr[d], nc=c+dc[d];
        if (inBounds(nr,nc,R,C) && dist[nr][nc]==-1 && grid[nr][nc]!='#') {
            dist[nr][nc] = dist[r][c] + 1;
            q.push({nr, nc});
        }
    }
}
```

GRID DFS – mark/explore a connected region

```
vector<vector<bool>> visited(R, vector<bool>(C, false));
void dfsGrid(int r, int c) {
    if (!inBounds(r,c,R,C) || visited[r][c] || grid[r][c]=='#') return;
    visited[r][c] = true;
    for (int d=0;d<4;d++) dfsGrid(r+dr[d], c+dc[d]);
}
```

===== READ GRID INPUT (common format) =====

```
vector<string> grid(R);
for (int i=0;i<R;i++) cin >> grid[i]; // grid[r][c] access, chars not ints
```

===== 8-DIRECTIONAL MOVEMENT (diagonals included) =====

```
int dr8[] = {-1,-1,-1,0,0,1,1,1};
int dc8[] = {-1,0,1,-1,1,-1,0,1};
```

===== START DFS/BFS FROM EVERY CELL =====

```
for(int r=0;r<R;r++)
for(int c=0;c<C;c++)
    if(!visited[r][c]) dfsGrid(r,c);
```

GRID DP – the bridge between DP and graphs

// dp[r][c] depends on dp[r-1][c] and/or dp[r][c-1] (values from above/left)
// Same 'skip or take' DP mindset from Page 18, applied to 2D movement instead
// of a 1D array – only right/down moves means no cycles, so DP applies cleanly.

DEMO 1: MIN PATH SUM (top-left to bottom-right, right/down only)

Use when: minimizing/maximizing a cost while moving only right or down.

```
vector<vector<long long>> dp(R, vector<long long>(C, 0));
dp[0][0] = grid[0][0];
for (int c=1; c<C; c++) dp[0][c] = dp[0][c-1] + grid[0][c]; // first row: only from left
for (int r=1; r<R; r++) dp[r][0] = dp[r-1][0] + grid[r][0]; // first col: only from above
for (int r=1; r<R; r++)
    for (int c=1; c<C; c++)
        dp[r][c] = grid[r][c] + min(dp[r-1][c], dp[r][c-1]);
// answer = dp[R-1][C-1]
```

DEMO 2: COUNT UNIQUE PATHS (right/down only, no costs)

```
vector<vector<long long>> dp(R, vector<long long>(C, 0));
for (int r=0; r<R; r++) dp[r][0] = 1; // only 1 way along first column
for (int c=0; c<C; c++) dp[0][c] = 1; // only 1 way along first row
for (int r=1; r<R; r++)
    for (int c=1; c<C; c++)
        dp[r][c] = dp[r-1][c] + dp[r][c-1]; // sum of ways, not min
// answer = dp[R-1][C-1]
```

DEMO 3: UNIQUE PATHS WITH OBSTACLES

Same as Demo 2, but blocked cells contribute 0 ways.

```
vector<vector<long long>> dp(R, vector<long long>(C, 0));
dp[0][0] = (grid[0][0] == 1) ? 0 : 1; // 1 = obstacle
for (int r=0; r<R; r++)
    for (int c=0; c<C; c++) {
        if (grid[r][c] == 1) { dp[r][c] = 0; continue; }
        if (r>0) dp[r][c] += dp[r-1][c];
        if (c>0) dp[r][c] += dp[r][c-1];
    }
```

SPACE OPTIMIZATION NOTE

// if dp[r][c] only needs row r-1, keep just ONE row array and overwrite in place
// (same rolling-variable idea as Page 18, extended to a 1D row instead of scalars)

===== READ GRID OF NUMBERS INTO 2D VECTOR =====

```
vector<vector<long long>> grid(R, vector<long long>(C));
for (int i=0; i<R; i++)
    for (int j=0; j<C; j++)
        cin >> grid[i][j];
```

===== FORMULA: number of monotonic paths (no obstacles), combinatorial =====

```
// paths from (0,0) to (R-1,C-1) moving only right/down = C(R+C-2, R-1)
// useful for verifying your DP answer, or when R,C large and no obstacles exist
// ===== COMMON GRID DP TRANSITION =====
// dp[r][c] usually combines values from top and left
// first row/column usually initialized separately
```

8 Directions

```
int dx8[] = {-1,-1,-1,0,0,1,1,1};
int dy8[] = {-1,0,1,-1,1,-1,0,1};
```

TOPOLOGICAL SORT (Kahn's Algorithm, BFS-based)

Use when: ordering tasks with dependencies (DAG only – no cycles).

```
vector<int> indeg(n, 0);
for (int u=0;u<n;u++) for (int v : adj[u]) indeg[v]++;
queue<int> q; for (int i=0;i<n;i++) if (indeg[i]==0) q.push(i);
vector<int> order;
while (!q.empty()) {
    int u = q.front(); q.pop(); order.push_back(u);
    for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
}
// if order.size() < n → graph has a cycle, no valid ordering exists
```

DSU (Disjoint Set Union / Union-Find)

Use when: grouping elements, checking connectivity – nearly O(1) per operation.

```
vector<int> parent, rnk;
void dsuInit(int n) { parent.resize(n); iota(parent.begin(),parent.end(),0);
    rnk.assign(n,0); }
int find(int x) { return parent[x]==x ? x : parent[x]=find(parent[x]); } // path
compression
void unite(int a, int b) {
    a = find(a); b = find(b); if (a == b) return;
    if (rnk[a] < rnk[b]) swap(a, b);
    parent[b] = a; if (rnk[a] == rnk[b]) rnk[a]++; // union by rank
}
bool connected(int a, int b) { return find(a) == find(b); }
```

DIJKSTRA (shortest path, non-negative weights)

Use when: shortest path in a weighted graph, all weights >= 0.

```
vector<long long> dist(n, LLONG_MAX);
priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
dist[src] = 0; pq.push({0, src});
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue; // outdated entry, skip
    for (auto [v, w] : wadj[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w; pq.push({dist[v], v});
        }
    }
}
// dist[v] = shortest distance from src, LLONG_MAX if unreachable
```

===== DSU: COUNT NUMBER OF DISTINCT GROUPS =====

```
set<int> roots;
for (int i=0;i<n;i++) roots.insert(find(i));
int numGroups = roots.size();
```

===== DIJKSTRA: READ WEIGHTED GRAPH INPUT =====

```
vector<vector<pair<int,int>>> wadj(n);
for (int i=0;i<m;i++) {
    int u,v,w; cin>>u>>v>>w; u--; v--;
    wadj[u].push_back({v,w});
    wadj[v].push_back({u,w}); // omit 2nd line if directed
}
```

===== DSU UNION =====

```
// if(find(a)!=find(b)) unite(a,b);
```


EULER'S PHI FUNCTION – $\phi(n)$ = count of $x \leq n$ coprime to n

```
int phi(int n) {
    int result = n;
    for (int p=2; (long long)p*p<=n; p++) {
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            result -= result / p;
        }
    }
    if (n > 1) result -= result / n;
    return result;
}
```

LCS – Longest Common Subsequence: $dp[i][j]$ = LCS of $a[0..i)$, $b[0..j)$

```
// match:  $dp[i][j]=dp[i-1][j-1]+1$  | no match:  $dp[i][j]=\max(dp[i-1][j], dp[i][j-1])$ 
vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
for (int i=1; i<=n; i++) for (int j=1; j<=m; j++)
     $dp[i][j] = (a[i-1]==b[j-1]) ? dp[i-1][j-1]+1 : \max(dp[i-1][j], dp[i][j-1]);$ 
// answer =  $dp[n][m]$ 
```

LIS – Longest Increasing Subsequence, $O(n \log n)$ via lower bound

```
vector<int> tails;
for (int x : a) {
    auto it = lower_bound(tails.begin(), tails.end(), x); // use upper_bound for
    // non-decreasing
    if (it == tails.end()) tails.push_back(x); else *it = x;
}
// answer = tails.size() (tails itself is NOT the actual LIS, only its length)
```

EDIT DISTANCE – min ops (insert/delete/replace) to turn a into b

```
vector<vector<int>> dp(n+1, vector<int>(m+1));
for (int i=0; i<=n; i++)  $dp[i][0] = i;$ 
for (int j=0; j<=m; j++)  $dp[0][j] = j;$ 
for (int i=1; i<=n; i++) for (int j=1; j<=m; j++) {
    if ( $a[i-1]==b[j-1]$ )  $dp[i][j] = dp[i-1][j-1];$ 
    else  $dp[i][j] = 1 + \min(\{dp[i-1][j-1], dp[i-1][j], dp[i][j-1]\});$  // replace,
    // delete, insert
}
// answer =  $dp[n][m]$ 
```

INTERVAL & SCHEDULING GREEDY – sort by end time, pick greedily

```
sort(intervals.begin(), intervals.end(), [](auto&a, auto&b){ return a.second<b.second;
});
int count = 0; long long lastEnd = LLONG_MIN;
for (auto& [s, e] : intervals) {
    if (s >= lastEnd) { count++; lastEnd = e; } // non-overlapping, greedily extend
}
// count = max number of non-overlapping intervals selectable
// key note: sorting by END time (not start) is what makes this greedy provably
// optimal
```

// $\phi(n)=n*\prod(1-1/p)$ over distinct prime factors

// LIS DP: $dp[i]=1+\max(dp[j])$ ($a[j]<a[i]$)

// lower_bound -> strict LIS

// upper_bound -> non-decreasing LIS

// LCS/Edit Distance -> backtrack from $dp[n][m]$ to recover answer